

Formal Languages

Capstone Project

Goal of the task

In this project, students will investigate a prominent NP-complete or NP-intermediate problem. The goal is to understand the problem's complexity, experiment with solution algorithms, and reflect on theoretical versus practical difficulty. This project combines theoretical reasoning, algorithm implementation, and data analysis.

1 Organization and management

All practical assessment tasks will be carried out in teams of three/four members. Teams will remain unchanged throughout the duration of the course.

2 Instructions

1. **Problem selection:** Each team is assigned a problem taken from the list provided at the end of this document. Should we wish to change problem, please, talk to your course coordinator.
2. **Problem description:** Describe the problem and explain why it is NP-complete or NP-intermediate.
3. **Algorithm Exploration:** Implement an exact solution that works for small instances. In addition, implement a heuristic or approximation that works for larger instances. Run experiments and collect basic results (runtime, solution quality, etc.).
4. **Analysis and Reflection:** Compare the theoretical complexity with observed performance. Reflect on why the problem is hard and which methods work best in practice.

3 Deliverables

- **Report (5 pages max):** Problem description, complexity classification, algorithms, experiments, and reflections.
- **Code:** Clearly documented and runnable.
- **Presentation (15 min) plus questions (10 min) :** Highlight main findings and insights.

4 Evaluation

Grading will be done in two steps. First, the written report and code, which will be submitted by Week 10, will receive a binary mark, namely satisfactory or unsatisfactory. Satisfactory reports will be invited to present their work orally, during Weeks 13 and 14. Oral presentations will receive points based on the following criteria.

- Understanding of the Problem.
- Understanding of the complexity class the problem belongs to.
- Description of reductions, proof sketches, or reasoning about hardness.
- Algorithmic Approach, including why chosen and how they work.
- Interpret experimental outcomes.
- Presentation Clarity & Communication

The overall mark of the Capstone project will be that given to the presentation. Unsatisfactory reports receive no marks and will not receive an invitation to present.

5 Problem set

1. **SAT:** Decide if a Boolean formula is satisfiable. This problem is relevant in Hardware/software verification, AI planning. More info can be found at https://en.wikipedia.org/wiki/Boolean_satisfiability_problem

2. **0-1 Knapsack:** Select items with maximum value under weight constraint. This problem is relevant in Resource allocation, cargo loading. More info can be found at https://en.wikipedia.org/wiki/Knapsack_problem
3. **Subset Sum / Partition:** Decide if a subset sums to target value. This problem is relevant in Cryptography, finance. More info can be found at https://en.wikipedia.org/wiki/Subset_sum_problem
4. **Vertex Cover:** Find small sets covering edges vertices. This problem is relevant in Network monitoring, facility location. More info can be found at https://en.wikipedia.org/wiki/Vertex_cover
5. **Set Cover:** Cover a universe with fewest sets. This problem is relevant in Sensor placement, test case selection. More info can be found at https://en.wikipedia.org/wiki/Set_cover_problem
6. **Traveling Salesman Problem (TSP):** Decide if a tour visits all cities within bound. This problem is relevant in Logistics, routing. More info can be found at https://en.wikipedia.org/wiki/Travelling_salesman_problem
7. **Job Scheduling / Makespan Minimization:** Schedule jobs on machines within time limit. This problem is relevant in Manufacturing, cloud computing. More info can be found at https://en.wikipedia.org/wiki/Job_shop_scheduling
8. **Bin Packing:** Pack items into fewest bins. This problem is relevant in Shipping, memory allocation. More info can be found at https://en.wikipedia.org/wiki/Bin_packing_problem
9. **Graph Coloring:** Assign $\leq k$ colors to vertices with no adjacent same-color vertices. This problem is relevant in Timetabling, frequency assignment. More info can be found at https://en.wikipedia.org/wiki/Graph_coloring
10. **Steiner Tree:** Connect terminal nodes with minimal total edge weight. This problem is relevant in Network design, VLSI. More info can be found at https://en.wikipedia.org/wiki/Steiner_tree_problem

11. **Clique / Independent Set:** Find a clique of size $\geq k$ or independent set of size $\geq k$. This problem is relevant in Social network analysis, bioinformatics. More info can be found at https://en.wikipedia.org/wiki/Clique_problem
12. **Hamiltonian Cycle / Path:** Decide if a cycle/path visits every vertex exactly once. This problem is relevant in Genome assembly, routing problems. More info can be found at https://en.wikipedia.org/wiki/Hamiltonian_path
13. **3-Dimensional Matching / Exact Cover (X3C):** Find a matching covering all elements. This problem is relevant in Combinatorial design, database joins. More info can be found at https://en.wikipedia.org/wiki/3-dimensional_matching
14. **Feedback Vertex Set / Feedback Arc Set:** Remove vertices/arcs to make graph acyclic. This problem is relevant in Deadlock avoidance, circuit optimization. More info can be found at https://en.wikipedia.org/wiki/Feedback_vertex_set
15. **Set Packing:** Find disjoint sets from a collection. This problem is relevant in Task assignment, resource allocation. More info can be found at https://en.wikipedia.org/wiki/Set_packing
16. **Closest String / Longest Common Subsequence:** Decide if a string exists within Hamming distance d of all given strings or subsequence of length $\geq k$. This problem is relevant in Bioinformatics, DNA sequencing. More info can be found at https://en.wikipedia.org/wiki/Longest_common_subsequence_problem
17. **Latin Square Completion / Sudoku:** Complete a partially filled Latin square or Sudoku. This problem is relevant in Constraint solving, AI puzzles. More info can be found at https://en.wikipedia.org/wiki/Latin_square
18. **Subset Sum variants in Cryptography:** Decide if subset sums meet target criteria. This problem is relevant in Knapsack cryptosystems, teaching NP-hardness. More info can be found at https://en.wikipedia.org/wiki/Subset_sum_problem

19. **Partition Into Triangles / Triangle Packing:** Partition graph vertices into disjoint triangles. This problem is relevant in Combinatorial research. More info can be found at https://en.wikipedia.org/wiki/Triangle_packing_problem
20. **Graph Partition / Minimum Bisection:** Partition graph into equal parts with minimal cut. This problem is relevant in Network reliability, clustering. More info can be found at https://en.wikipedia.org/wiki/Graph_partition
21. **Clique Cover / Domatic Number:** Partition vertices into cliques or domatic sets. This problem is relevant in Theoretical combinatorics. More info can be found at https://en.wikipedia.org/wiki/Clique_cover
22. **Path Cover / Partition into Paths:** Cover all vertices with paths. This problem is relevant in Graph theory research. More info can be found at https://en.wikipedia.org/wiki/Path_cover
23. **Exact Cover / Hypergraph Partitioning:** Find exact partitions in hypergraphs. This problem is relevant in Combinatorial design. More info can be found at https://en.wikipedia.org/wiki/Exact_cover
24. **Integer Factorization:** Decide if an integer can be factored into two non-trivial integers. This problem is relevant in Cryptography (RSA). More info can be found at https://en.wikipedia.org/wiki/Integer_factorization
25. **Discrete Logarithm:** Find x such that $g^x = h$ modulo p . This problem is relevant in Cryptography (Diffie-Hellman). More info can be found at https://en.wikipedia.org/wiki/Discrete_logarithm
26. **Quadratic Residuosity:** Decide if a number is a quadratic residue modulo n . This problem is relevant in Cryptography (Goldwasser-Micali). More info can be found at https://en.wikipedia.org/wiki/Quadratic_residuosity_problem
27. **Graph Isomorphism:** Decide if two graphs are isomorphic. This problem is relevant in Chemistry, pattern recognition. More info can be found at https://en.wikipedia.org/wiki/Graph_isomorphism_problem

28. **Integer Lattice Problems (SVP / CVP):** Shortest or closest vector in a lattice. This problem is relevant in Post-quantum Cryptography. More info can be found at https://en.wikipedia.org/wiki/Shortest_vector_problem